

Used in filters like sort by
name/date

LECTURE - II

Wednesday
30/9/26.

□ Radix Sort: Sort d -digit numbers by radix/base value.
* we need a "stable sort" subroutine - (Counting Sort)

- ① LSD : start from least significant to most signif.
- ② MSD : start from most sig. to least signif.

maintains numerical order of data

maintains ~~alphabetical~~ alphabetical / lexicographical order.

$n \rightarrow$ no. of data values like: 456 $\rightarrow n=2$

$d \rightarrow$ no. of digits in each value $\rightarrow d=3$

$k \rightarrow$ highest digit in that specific base of ~~base~~ values i.e. 10 (decimal)
 $\therefore k = 10 - 1 = 9$ Date _____

Code:

Radix Sort (A[1:n], n, d)

for ($i \leftarrow 1$ to d)

// $O(d)$

Sort each column i by using a stable sort. // $O(n+k)$
 $\rightarrow$ Counting Sort.

$\therefore$ Radix Sort $\rightarrow T(n) = O(d(n+k))$

Example:

4 3 2 1
4 5 1 3

9 9 9 9

1 3 4 2

5 6 2 2

LSD $\rightarrow$

1 3 4 2

5 6 2 2

4 5 1 3

9 9 9 9

$\rightarrow$ 1st column from
least sig. side
sorted

2nd column from right

4 5 1 3

5 6 2 2

1 3 4 2

9 9 9 9

3rd Column $\rightarrow$

1 3 4 2

4 5 1 3

5 6 2 2

9 9 9 9

4th
Column $\rightarrow$

1 3 4 2
4 5 1 3
5 6 2 2
9 9 9 9

MSD: After sorting 1st column from left (msb);
we'll make bins which will be on the basis of Most
sig. number (i.e. left most).

If data isn't sorted in any specific ~~bin~~ bins; we'll further make bins
on the basis of next column that'll be 2nd after 1st column (LSB)

4 5 1 3 $\rightarrow$ 3 2 5 1 $\rightarrow$ 3 2 5 1 sorted bin

6 1 1 1

4 5 1 3

4 1 3 2

1 1 3 2

4 5 2 1

$\rightarrow$

4 5 2 1

$\rightarrow$

4 5 1 3

$\rightarrow$

4 5 1 3

..... sorted bin

5 5 1 3

4 1 3 2

4 5 2 1

4 5 2 1

6 8 1 4

5 5 1 3

$\rightarrow$

5 5 1 3

..... sorted bin

4 1 3 2

6 1 1 1

6 1 1 1

..... sorted bin

3 2 5 1

6 8 1 4

6 8 1 4

..... sorted bin

} Tesay he
sort hojaye. STOP
Don't go for
Sorting of next
column.

Radix = 2 (Bits):

Arr = [011010, 011100, 110010, 111000]

- $n = 4$ (no. of numbers in array)
- $b = 6$ (no. of bits in each data value) (111000)

Since we don't want to do 1 bit check at a time for 6 separate bits $\therefore$ radix sort make chunks of d -digits and that chunk is of length $= r$

- Suppose $r = 3$ (splits 6 bit no. in half of 3-3 bits) ($r \leq b$)
- $d = 2$ which is no. of new numbers made after splitting in d chunks by r values.

$$d = b/r$$

- Max. decimal value that can fit in that chunk of r -bits is 'k'

$$k = 2^r - 1 \quad \rightarrow 2^3 - 1 = 7 = (111)$$

⇒ Time Complexity of Radix Sort:

Every pass takes 'n' times and possibility for each 'b' is 2^r .

$$\therefore (n + 2^r).$$

Since this process is repeated for d-number which is b/r ...

$$\boxed{\text{Total } T(n) = (b/r) \cdot (n + 2^r)}$$

When does this becomes $O(n)$?

For 'n' data suppose $b = \log_2(n)$ and $r \approx \log_2(n)$ as 'r'

could be less than equal to 'b'. Then; the factor of $b/r = \text{constant}$

$$\bullet \frac{b}{r} = \frac{c \cdot \log_2(n)}{\log_2(n)} = c (\text{constant}) \quad \therefore \text{ignored}$$

$$\bullet n + 2^{(\log_2(n))} = n + n = \boxed{O(n)}.$$

Notes: This is useful when no. of data values is very large (i.e.: n) and k is relatively smaller.

NOTE: If we've no extra space/array, we'll use Quick sort as it's an inplace algo. (which means no. extra memory & original array is altered) whose avg. is " $n \log n$ ".

• If we have extra memory, then we can use Radix sort which is an out of place algo (extra memory/array made) whose avg. is $O(n)$

Date _____